

Face the World: Frontend & Nginx

2026-04-19 · brisk pomegranate Haubentaucher

BROWSERS DO NOT TALK TO YOUR APP SERVER DIRECTLY.

The Problem

UVICORN LISTENS ON PORT 8000, but browsers expect port 80 (HTTP) and 443 (HTTPS). You need something in front that handles the ports, serves static files, and forwards API requests. That something is Nginx.

The Frontend Stack

Why Vanilla JavaScript

HTML FOR STRUCTURE, CSS FOR LAYOUT, JAVASCRIPT FOR INTER-ACTIVITY. The fetch API sends requests to the backend.

THE STARTER CODE IS PROVIDED. Students need to understand it well enough to wire it up and extend it in Block 10. The key pieces:

Drawing canvas A 28×28 pixel grid where users draw with the mouse.

Submit button Extracts the pixel array from the canvas and sends a POST /api/classify request.

Results panel Displays the predicted digit, confidence score, and a confidence bar.

THE fetch CALL is the bridge between frontend and backend:

```
async function classifyDrawing() {
  const pixels = getPixelsFromCanvas(); // 28x28 array
  const response = await fetch("/api/classify", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-API-Key": API_KEY
    },
    body: JSON.stringify({ pixels: pixels })
  });
  const result = await response.json();
  displayResult(result);
}
```

MDN Web Docs (HTML/CSS/JS reference): <https://developer.mozilla.org/>

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Why not React, Vue, or Svelte? Frameworks solve problems that appear at scale, component reuse, state management, routing across dozens of views. PixelWise has one page with one canvas and one results panel. A framework would add a build toolchain (Node.js, npm, bundler), a learning curve orthogonal to this course, and complexity that hides what is actually happening between browser and server. Vanilla JS keeps the HTTP layer visible.

No build step, no node_modules, no framework version to maintain. The code runs in any browser, can be read by any developer, and deploys as static files served directly by Nginx. Students who later build larger applications will recognise when a framework becomes necessary, and why.

Exercise: Deploy the Frontend

COPY THE PROVIDED STARTER CODE to frontend/ on the server.

Walk through the code to understand: the fetch call to /api/classify, the pixel array extraction from the canvas, the response rendering.

On the server

ls /opt/pixelwise/frontend/

Should contain: index.html, style.css, app.js

EXERCISES are for practice and reinforcement. If your VM is broken, restore the B6-complete snapshot. At the end of this session, take a snapshot named B7-complete.

Read app.js top to bottom. Locate the fetch call, the canvas event handlers, and the function that turns canvas pixels into the JSON payload. If the bridge between browser and backend is to make sense in the next exercise, it must first be visible to you here.

Nginx: Three Roles in One

NGINX SERVES THREE ROLES in the PixelWise stack:

Static file server Serves HTML, CSS, and JavaScript files far more efficiently than Python.

Reverse proxy Forwards `/api/` requests to `localhost:8000` (uvicorn). The browser never talks to uvicorn directly.

SSL terminator Handles HTTPS encryption. The backend receives plain HTTP; encryption is Nginx's job.

Nginx: <https://nginx.org/>
Nginx Docs: <https://nginx.org/en/docs/>

Nginx Config Anatomy

NGINX CONFIGURATION is structured in nested blocks:

```

server {
    listen 80;
    server_name 192.168.56.11;

    # Serve static files
    location / {
        root /opt/pixelwise/frontend;
        index index.html;
    }

    # Proxy API requests to uvicorn
    location /api/ {
        proxy_pass http://localhost:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

The `proxy_pass` directive is the reverse proxy: any request to `/api/classify` is forwarded to `http://localhost:8000/classify`. The browser sees `https://192.168.56.11/api/classify`; uvicorn sees `http://localhost:8000/classify`. The trailing slash matters, it strips the `/api` prefix.

Exercise: Install and Configure Nginx

INSTALL NGINX on the server:

```
sudo apt install -y nginx
```

CREATE THE PIXELWISE SITE CONFIGURATION:

```
sudo nano /etc/nginx/sites-available/pixelwise
```

Use the config from the theory above. Enable the site and restart:

```
sudo ln -s /etc/nginx/sites-available/pixelwise \
    /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t          # test config syntax
sudo systemctl restart nginx
```

VERIFY THE REVERSE PROXY. Open a browser on the host machine and navigate to <http://192.168.56.11>. You should see the PixelWise frontend. Draw a digit and submit, the request goes through Nginx to uvicorn and back. The browser sees a single origin; uvicorn never appears in any URL.

nginx -t parses the configuration without applying it. Run it before every restart; a syntax error caught here is cheaper than a daemon that refuses to start in production.

CORS: Cross-Origin Resource Sharing

CORS IS A BROWSER SECURITY POLICY that blocks requests from one origin to another. When your frontend is served on port 443 and the API runs on port 8000, the browser considers them different origins and blocks the request.

DURING DEVELOPMENT, CORS causes pain. The browser's console shows a cryptic error: Access to fetch has been blocked by CORS policy. The fix: configure FastAPI to allow requests from the frontend's origin:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://192.168.56.11"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

WITH NGINX AS A REVERSE PROXY, CORS often resolves itself: the frontend and API share the same origin (<https://192.168.56.11>), so the browser does not block the request. But students must understand CORS because they will hit it in every deployment that separates frontend and backend.

CORS explained: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Never use `allow_origins=["*"]` in production, it allows any website to call your API. Configure allowed origins explicitly.

Exercise: Debug CORS

IF THE FRONTEND CANNOT REACH THE API, check the browser console for CORS errors. Open the developer tools, switch to the console tab, and reload the page; a blocked request prints a message beginning with Access to fetch has been blocked by CORS policy.

ADD THE CORS MIDDLEWARE to FastAPI with the correct allowed origin:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://192.168.56.11"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Restart uvicorn and reload the page. The console should now be silent for the previously blocked request.

This is a rite of passage. Every developer who has shipped a frontend separated from its backend has read this error message at least once. Knowing the cause makes the cure a one-line edit instead of an afternoon of bafflement.

URL Design

CLEAN, READABLE URLs matter:

Good /results?digit=7&model=v2

Bad /getResults.php?action=list&d=7

QUERY PARAMETERS serve filtering and configuration. Storing session IDs in URLs (/classify?session=abc123) lets you track a user's session without cookies, link to specific results, or pass configuration to the frontend. The URL is state, treat it deliberately.

HTTPS and TLS

WITHOUT HTTPS, API KEYS TRAVEL IN PLAINTEXT. Anyone on the network can read them. HTTPS encrypts the entire channel between browser and server.

Self-signed certificate Fine for development. The browser shows a warning, ignore it.

Let's Encrypt Free, automated certificates for production (<https://letsencrypt.org/>). Renewal is automatic via certbot (<https://certbot.eff.org/>).

GENERATING A SELF-SIGNED CERTIFICATE:

```
sudo openssl req -x509 -nodes -days 365 \
    -newkey rsa:2048 \
    -keyout /etc/ssl/private/pixelwise.key \
    -out /etc/ssl/certs/pixelwise.crt
```

Security Headers

NGINX CAN ADD SECURITY HEADERS to every response, one place, all responses covered:

```
add_header X-Frame-Options "DENY";
add_header X-Content-Type-Options "nosniff";
add_header Content-Security-Policy "default-src 'self'";
```

Exercise: Enable HTTPS and Security Headers

GENERATE A SELF-SIGNED CERTIFICATE on the server:

```
sudo openssl req -x509 -nodes -days 365 \  
    -newkey rsa:2048 \  
    -keyout /etc/ssl/private/pixelwise.key \  
    -out /etc/ssl/certs/pixelwise.crt
```

UPDATE THE NGINX CONFIG to listen on port 443 with the certificate:

```
server {  
    listen 443 ssl;  
    ssl_certificate /etc/ssl/certs/pixelwise.crt;  
    ssl_certificate_key /etc/ssl/private/pixelwise.key;  
    # ... rest of config  
}
```

Restart Nginx and verify `https://192.168.56.11` works (accept the browser warning).

ADD SECURITY HEADERS to the Nginx config: `X-Frame-Options`, `X-Content-Type-Options`, and `Content-Security-Policy`. Restart and verify they appear in the response headers:

```
curl -I https://192.168.56.11 -k
```

The `-k` flag tells `curl` to accept the self-signed certificate without complaint. In production with a Let's Encrypt certificate, you would not need it; the warning disappears and the headers are the same.

Optional: ufw

THE FIREWALL IS THE OUTER SKIN OF THE SERVER. On a host-only network there is no public internet to defend against, so the lecture VMs run without one. On a real production host the rule of thumb is: deny by default, allow exactly the ports the service needs. `ufw` expresses this in a few commands and stays out of the way once configured.

`ufw` (Uncomplicated Firewall) is Ubuntu's firewall tool. On managed production servers, the firewall is typically preconfigured to allow only ports 22 (SSH), 80 (HTTP), 443 (HTTPS). On the classroom VMs this is a formality (host-only network), but students should know the concept exists.

Exercise: Commit and Snapshot

COMMIT THE FRONTEND AND NGINX WORK:

```
git add frontend/ nginx/  
git commit -m "Add frontend and Nginx reverse proxy"  
git push
```

TAKE A SNAPSHOT of both VMs. Name them B7-complete. This is your known good state with the full stack live: frontend, reverse proxy, HTTPS, and security headers all in place.

A NOTE ON WHAT COMES NEXT. The full stack now answers the browser, but every step you took to get here was manual: copying files, editing configs, restarting services. The next block automates that path with deployment scripts so a single command takes code from dev to prod without you remembering anything.

Security Thread

SECURITY HEADERS ARE CONFIGURED AT THE NGINX LAYER, one place, all responses covered. HTTPS encrypts the channel; without it, API keys travel in plaintext. CORS misconfiguration is a common deployment bug, configure allowed origins explicitly, never * in production.

Self-Reflection and Recap

REFLECTION QUESTIONS:

- Why do browsers not talk to uvicorn directly? What does Nginx add?
- What is a reverse proxy and why is it called “reverse”?
- What happens if you set `allow_origins=["*"]` in production?
- Why do we use vanilla JavaScript instead of a framework like React?
- What is the difference between a self-signed certificate and one from Let's Encrypt?

MILESTONE: PixelWise accessible in a browser at `https://192.168.56.11`. Draw a digit, get a prediction. Nginx serves static files and proxies the API. The full stack is live: browser → Nginx → uvicorn → FastAPI → PostgreSQL.